

Practical 8

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc()
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        a[i] = i + 1;
    }

    printf("malloc memory: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    // calloc()
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc memory: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }
    printf("\n");
```

```
// realloc()
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL) {
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++) {
    a[i] = i + 1;
}

printf("After realloc: ");
for (i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}
printf("\n");

// free()
free(a);
free(b);

printf("Memory freed successfully.\n");

return 0;
}
```

```
adithya@LAPTOP-07A78KDI: ~  
adithya@LAPTOP-07A78KDI:~/OS/OS$ nano memory.c  
adithya@LAPTOP-07A78KDI:~/OS/OS$ gcc -Wall -g memory.c -o memory  
adithya@LAPTOP-07A78KDI:~/OS/OS$ ./memory  
malloc memory: 1 2 3 4 5  
calloc memory: 0 0 0 0 0  
After realloc: 1 2 3 4 5 6 7 8 9 10  
Memory freed successfully.  
adithya@LAPTOP-07A78KDI:~/OS/OS$ valgrind --version  
valgrind-3.26.0  
adithya@LAPTOP-07A78KDI:~/OS/OS$ valgrind --leak-check=full --show-leak-kinds=all ./memory  
==1392== Memcheck, a memory error detector  
==1392== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.  
==1392== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info  
==1392== Command: ./memory  
==1392==  
malloc memory: 1 2 3 4 5  
calloc memory: 0 0 0 0 0  
After realloc: 1 2 3 4 5 6 7 8 9 10  
Memory freed successfully.  
==1392==  
==1392== HEAP SUMMARY:  
==1392==    in use at exit: 0 bytes in 0 blocks  
==1392== total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated  
==1392==  
==1392== All heap blocks were freed -- no leaks are possible  
==1392==  
==1392== For lists of detected and suppressed errors, rerun with: -s  
==1392== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
adithya@LAPTOP-07A78KDI:~/OS/OS$ valgrind --leak-check=full --show-leak-kinds=all ./memory  
==1402== Memcheck, a memory error detector  
==1402== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.  
==1402== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info  
==1402== Command: ./memory  
==1402==  
malloc memory: 1 2 3 4 5  
calloc memory: 0 0 0 0 0  
After realloc: 1 2 3 4 5 6 7 8 9 10  
Memory freed successfully.  
==1402==  
==1402== HEAP SUMMARY:  
==1402==    in use at exit: 0 bytes in 0 blocks  
==1402== total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated  
==1402==  
==1402== All heap blocks were freed -- no leaks are possible  
==1402==  
==1402== For lists of detected and suppressed errors, rerun with: -s  
==1402== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
adithya@LAPTOP-07A78KDI:~/OS/OS$ |
```

PART 2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main() {
    int *data;
    pid_t pid;

    // Allocate memory
    data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    // Initialize data
    for (int i = 0; i < SIZE; i++) {
        data[i] = i;
    }

    printf("Parent: Memory allocated and initialized.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild process created.\n");
```

```
printf("Child PID: %d\n", getpid());

printf("Child: Before modification data[0] = %d\n", data[0]);

// Modify memory
data[0] = 9999;

printf("Child: After modification data[0] = %d\n", data[0]);

printf("Child: Modified memory after fork.\n");

free(data);
exit(0);
}
else {
    // Parent process
    printf("\nParent: Child PID = %d\n", pid);

    sleep(2);

    printf("Parent: data[0] = %d\n", data[0]);

    wait(NULL);

    printf("Parent: Child process finished.\n");

    free(data);
}

return 0;
}
```

```
adithya@LAPTOP-07A78KDI:~/OS/OS$ nano cow.c
adithya@LAPTOP-07A78KDI:~/OS/OS$ gcc -Wall -g cow.c -o cow
adithya@LAPTOP-07A78KDI:~/OS/OS$ ./cow
Parent: Memory allocated and initialized.
Parent PID: 1442

Parent: Child PID = 1443

Child process created.
Child PID: 1443
Child: Before modification data[0] = 0
Child: After modification data[0] = 9999
Child: Modified memory after fork.
Parent: data[0] = 0
Parent: Child process finished.
adithya@LAPTOP-07A78KDI:~/OS/OS$ ps -o pid,ppid,rss,vsz,cmd
  PID    PPID   RSS   VSZ CMD
   352     351  5556   6268 -bash
  1455     352   4036   7112 ps -o pid,ppid,rss,vsz,cmd
adithya@LAPTOP-07A78KDI:~/OS/OS$ |
```